

INSTALACIÓN DEL ENTORNO DE TRABAJO.

Para trabajar durante el curso serán necesarias una serie de herramientas. Estas herramientas van a ir desde el control del código hasta el uso de un entorno que nos permita la escritura y prueba del código desarrollado. Para ello es necesario:

1. Un entorno de desarrollo. Un editor con opciones avanzadas para depuración y escritura de código: Visual Studio Code
2. Sistema de control de versiones. Usaremos GIT de forma local y Github en remoto.
3. Servidor de aplicaciones web. Usaremos docker y un contenedor con apache-php-xdebug para la ejecución de las aplicaciones
4. Depurador para php. Usaremos XDebug.

1.- Visual Studio Code

Visual Studio Code es un IDE de propósito general desarrollado por Microsoft y que nos permite desarrollar PHP añadiendo una serie de pluggins.

- **Spanish Language Pack:** Si queremos el entorno en español.



Spanish Language Pack for Visual Studio Code

Microsoft microsoft.com | 11.642.746 | ★★★★★ (15)

Language pack extension for Spanish

[Desinstalar \(todos los perfiles\)](#) ☒ Actualización automática

[DETALLES](#) [CARACTERÍSTICAS](#) [REGISTRO DE CAMBIOS](#)

Paquete de idioma español para VS Code

- **Php Intelliphense:** Es un asistente de código de PHP que nos facilita en gran medida la escritura de código PHP.



PHP IntelliSense

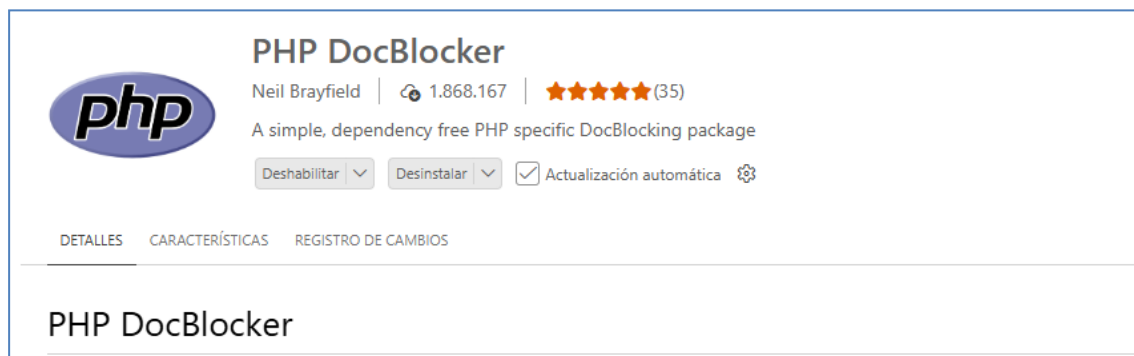
Ben Mewburn | 3.412.689 | ★★★★★ | Repositorio | License

PHP code intelligence for Visual Studio Code

[Deshabilitar](#) [Desinstalar](#) ☒ Esta extensión está habilitada globalmente.

Esta extensión se recomienda en función de los archivos abiertos recientemente.

- **PHP DocBlocker:** Asistente para añadir documentación a nuestro código



2.- Sistema de control de versiones.

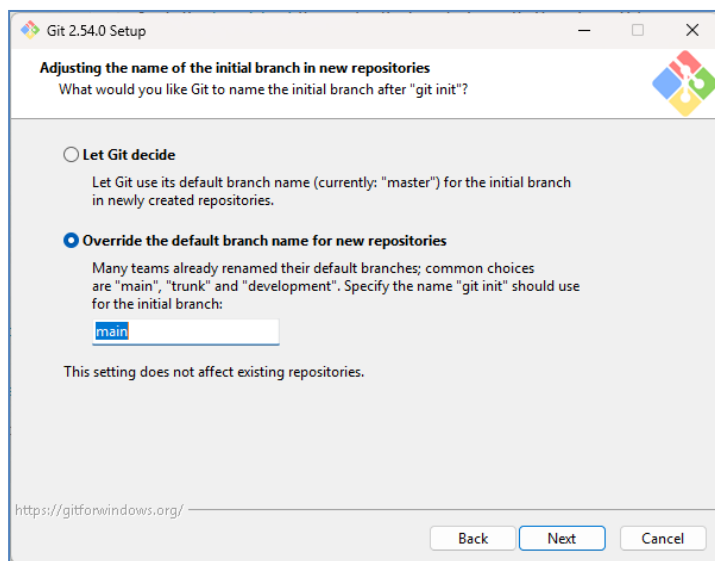
Un sistema de control de versiones nos permite mantener un historial del código de nuestras aplicaciones, pudiendo en cualquier momento consultar el código antiguo.

Usaremos GIT, un sistema distribuido muy utilizado en la industria actualmente.

Lo primero es instalar **GIT** a partir del binario que encontramos en <https://git-scm.com/install/windows>. Descargaremos la version para X64.

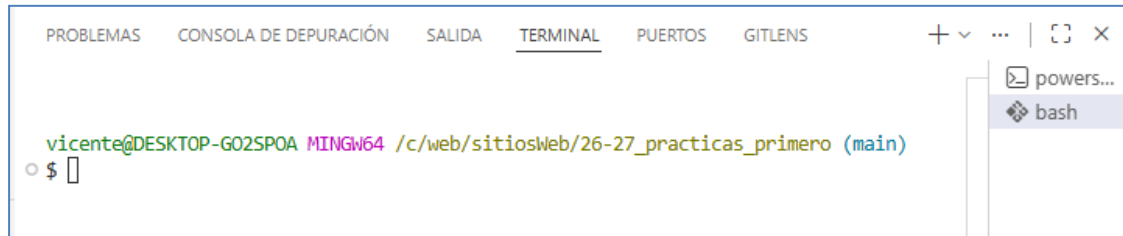
Lanzaremos el instalador y se irá dando a siguiente con las opciones por defecto que nos aparecen salvo con:

- El nombre de la rama inicial, que indicaremos como main



Git se maneja fundamentalmente mediante comandos. Para ello incorpora un interprete llamado bash. (C:\Program Files\Git\bin\bash.exe). Este interprete ayuda con la gestión de un repositorio de git dando información adicional.

Dentro de Visual Studio Code, tenemos acceso a bash en la pestaña Terminal

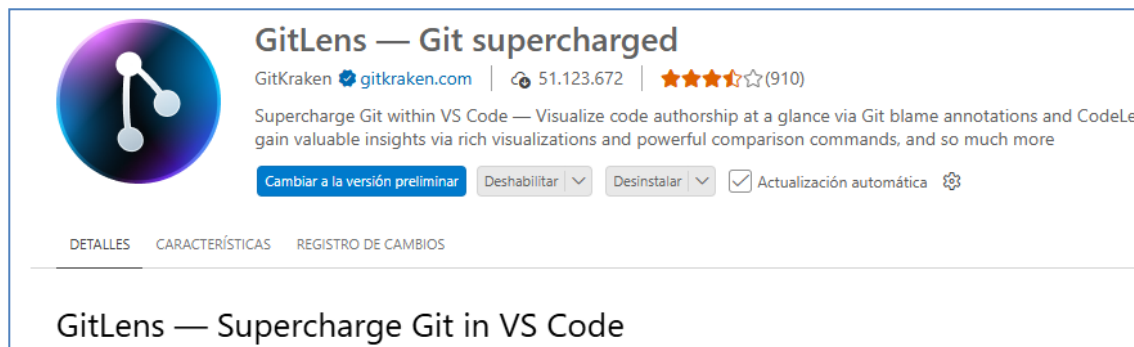


Una vez lo tenemos instalado procedemos a realizar una configuración inicial: Usamos el comando GIT para registrar nuestro nombre y correo electrónico:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-email@g.educaand.es"
```

Para el correo electrónico usaremos nuestra cuenta corporativa.

Visual Studio Code trae incorporado el soporte para Git. Únicamente añadiremos una extensión que nos da información adicional para los repositorios de Git: Gitlens.



A partir de aquí ya podemos crear nuestro repositorio, agregar archivos, hacer commits, etc. Ver manual **Git y Visual Studio**.

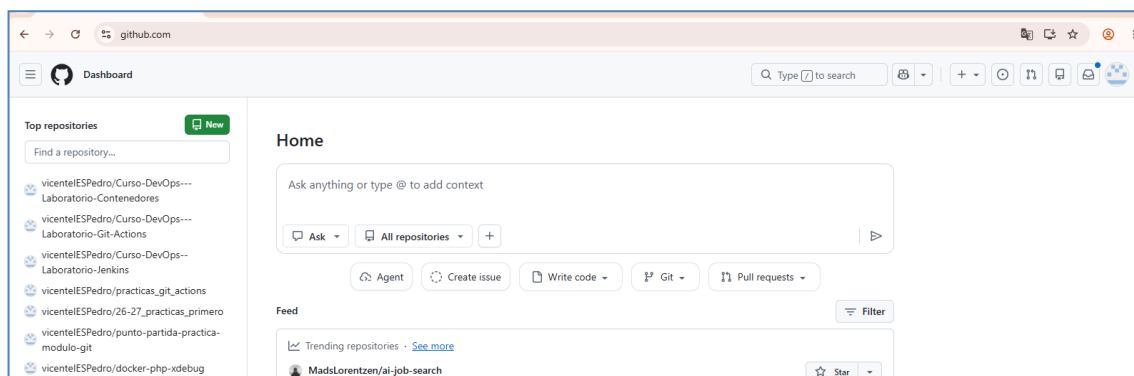
El siguiente paso, es usar **Github** para almacenar y gestionar nuestros repositorios.

Github es una aplicación web para Git. Permite el trabajo en equipo así como otra serie de funcionalidades muy usadas actualmente para el desarrollo de aplicaciones con metodología CI/CD

Lo primero es acceder a la página <https://github.com/> y a **sign up** para registrarnos. Nos vamos a registrar con la cuenta corporativa: tu-email@g.educaand.es.

Después de registrarnos accedemos con **sign in**.

Muestro mi pantalla inicial de github (cuenta **vicenteESPedro**, vtejtru288@g.educaand.es)



Ver manual **github**

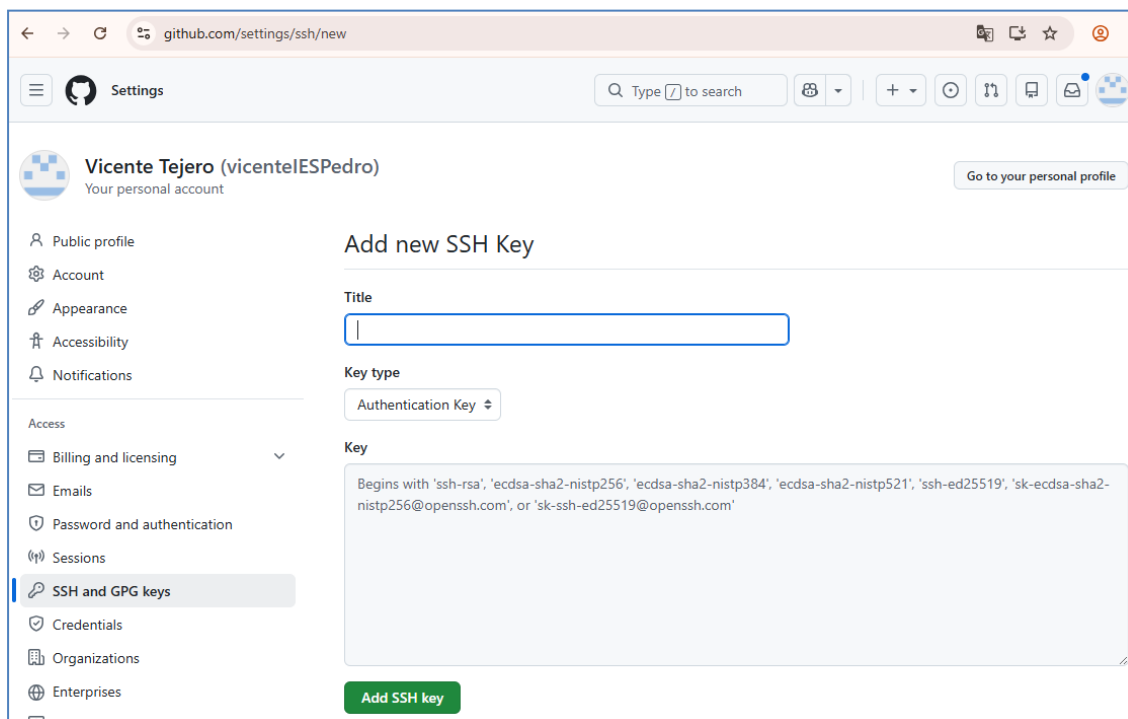
Para trabajar con GITHUB y poder acceder a nuestros repositorios existen dos métodos: usar usuario/contraseña o usar certificados.

Los certificados nos permiten trabajar directamente desde la línea de comandos. Usaremos este método para validarnos.

Creamos un certificado (clave pública/clave privada) mediante el comando ssh-keygen:

```
ssh-keygen -t ed25519 -C "tu-email@g.educaand.es"
```

Por defecto, nos colocará los ficheros del certificado (id_ed25519 y id_ed25519.pub) en el directorio “.ssh” de nuestra carpeta personal. El siguiente paso es registrar nuestra clave pública en github. Tenemos que irnos en github a settings/SSH and GPG keys/new SSH key



Para trabajar con git local y github, tenemos que crear un repositorio en github y enlazarlo a nuestro repositorio local de dos formas posibles:

- Añadir el repositorio remoto a un repositorio local ya existente:

```
git remote add nombre url-ssh
```

- Clonar el repositorio remoto en una carpeta local:

```
git clone url-ssh
```

A partir de aquí, cada vez que queramos actualizar el repositorio en github usaremos git push.
git push:

```
git push -u remoto rama-remota (para que quede enlazada la rama local con la remota, solo primera vez)
```

```
git push (veces posteriores)
```

3.- DOCKER

Para ejecutar las aplicaciones en php necesitamos un servidor de aplicaciones web. Normalmente apache-php-mysql. La primera solución es instalarnos un paquete al estilo XAMPP que trae todo incorporado.

Sin embargo, en la actualidad se tiende a trabajar usando máquinas virtuales donde tenemos instalado apache. Esto nos permite trabajar con diferentes versiones de apache o probar nuevos entornos sin que tengamos que hacerlo en nuestra máquina. Esto se consigue con docker, entorno que permite definir contenedores (máquinas virtuales) con un software específico.

Para instalar Docker, nos vamos a la dirección <https://www.docker.com/products/docker-desktop/> y descargamos la versión para AMD64. Es importante que instalemos WSL. WSL (Subsistema de Windows para Linux) es una utilidad que permite a Windows ejecutar una instalación de linux en paralelo. Esto es necesario ya que Docker corre sobre Linux.

Una vez instalado ver el manual **docker**

En Visual Studio Code instalaremos pluggins relacionados con docker:

- Docker

Para esta asignatura he creado un contenedor con apache. Podemos encontrarlo en mi “docker”

vicentett1. Una vez en ella podemos acceder al contenedor: php-xdebug

La ruta completa es: <https://hub.docker.com/r/vicentett1/php-xdebug>

Para trabajar con este contenedor podemos seguir los siguientes pasos:

- crear una carpeta para nuestro proyecto.
- Abrir Visual Studio y abrir la carpeta correspondiente.
- Irnos a terminal y abrir un terminal bash
- Ejecutar el comando docker: `docker run -d --name pro1 -p 8080:80 vicentett1/php-xdebug:1.0`
Con la que creamos el contenedor pro1, enlazamos el puerto 8080 del host al puerto 80 del contenedor y lo ejecutamos en segundo plano
- Nos “colocarnos en la máquina” con el comando docker: `docker exec -it pro1 bash`
- Ejecutamos los comandos que queramos directamente en la máquina Linux. Por ejemplo, para editar el archivo index.php, necesitamos editarlo usando nano (si no lo tenemos, lo instalamos):
`apt-get update`
`apt-get install nano`
`nano index.php`

Para poder trabajar localmente con el contenido de la carpeta raíz para el sitio, podemos usar el docker compose que indico en mi sitio de docker.

Los pasos serían:

- Crear el fichero docker-compose.yml en la carpeta raíz con el contenido:

services:

php:

image: `vicentett1/php-xdebug:1.0`

volumes:

- **type:** `bind`
source: `./web`
target: `/var/www/html`

extra_hosts:

- "host.docker.internal:host-gateway"

ports:

- 8080:80
- Ejecutar el fichero con el comando: docker compose up
- Editar en la carpeta ./web el contenido. Este será el contenido de nuestra aplicación web

4.- Herramienta de depuración

Xdebug es una utilidad que nos permite depurar scripts php tanto de forma local como accediendo al servidor. Esta utilidad nos permite depurar línea a línea, ver el valor de las variables, etc.

Ya que vamos a usar docker, necesitaremos de un contenedor que incorpore xdebug. En el apartado anterior ya se ha indicado como usar un contenedor con estas características.

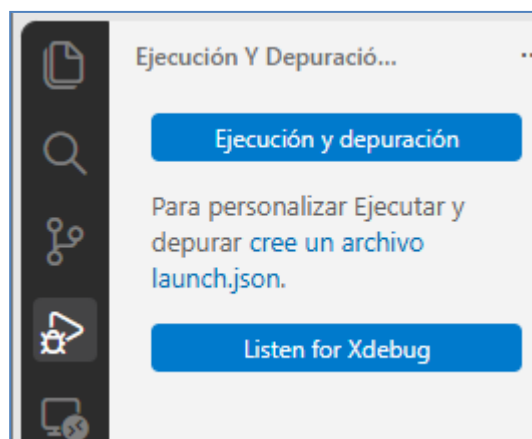
Lo que nos falta es configurar Visual Studio para poder depurar.

Debemos instalar el plugin PHP debug:



Una vez instalado, configuraremos Visual Studio para que se pueda depurar el código PHP:

- Creamos un archivo php en la carpeta web y lo abrimos.
- Nos vamos a la depuración en Visual Studio

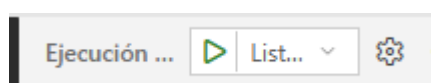


- Pulsamos en el botón Listen for Xdebug.

- Editamos la configuración para XDebug

- Añadimos la siguiente configuración:

```
{
    "name": "Listen for Xdebug",
```



pulsando en la

```
    "type": "php",  
    "request": "launch",  
    "port": 9003,  
    "pathMappings": {  
        "/var/www/html": "${workspaceRoot}/web"  
    }  
},
```

Lo importante es la línea pathMappings que mapea nuestra carpeta en el host a la carpeta en el contenedor.

- A partir de aquí ya se puede depurar.